

ROS2

DAY 3

Node Introspection, rqt & rqt_graph

Complete Beginner-Friendly

1. Introduction

By Day 2, you learned how to:

- Create ROS2 nodes (programs running in ROS2)
- Make nodes communicate using **topics** (publishers/subscribers)
- Send and receive messages

Day 3 goal:

- See **which nodes are running**
- Inspect **their topics and services**
- Visualize the **communication graph** using `rqt` and `rqt_graph`

Think of it like opening the hood of your robot to check the wiring and see if all parts are connected correctly.

2. Why Node Introspection Matters

Node introspection allows you to:

- Check if your nodes are actually running
- See what topics each node publishes or subscribes to
- Inspect services provided by nodes
- Debug communication issues

Connection to Day 1 & 2:

- In Day 1 & 2, you wrote nodes and topics.
- You assumed messages were flowing correctly.
- Day 3 shows the actual connections **in real-time**.

3. Node Introspection Commands

Before using any ROS2 commands, you must **open a terminal** and **source your ROS2 environment**:

1. Open a terminal on your computer.
2. Type the following command and press **Enter**:

```
source /opt/ros/<ros2_distro>/setup.bash
```

Replace `<ros2_distro>` with your ROS2 version (for example, `humble` or `iron`). This prepares your terminal to run ROS2 commands.

3.1 List all running nodes

1. In the terminal, type:

```
ros2 node list
```

2. Press **Enter**.

Expected output example:

```
/controller_node  
/motor_node  
/imu_node
```

- Each line represents a node currently running in ROS2.
- If your nodes from Day 1 & 2 are **not listed**, they are not active.

If a node is not active:

1. Open a **new terminal** and source ROS2 as above.
2. Launch your node using the appropriate launch file or command. For example, if you have a package called `day2_pkg` with a node `controller_node.py`:

```
ros2 run day2_pkg controller_node
```

3. Go back to the first terminal and run:

```
ros2 node list
```

4. Verify that the node now appears in the list.

3.2 Inspect a node

1. Choose a node from the list (for example `/controller_node`).
2. Type the following command in the terminal and press **Enter**:

```
ros2 node info /controller_node
```

3. You will see output similar to this:

Subscriptions:

```
/imu/data    sensor_msgs/msg/Imu  
/cmd_vel     geometry_msgs/msg/Twist
```

Publications:

```
/motor_cmd   std_msgs/msg/Float32
```

Services:

```
/reset_pid   std_srvs/srv/Empty
```

What it means:

- **Subscriptions** → topics the node listens to
- **Publications** → topics the node publishes
- **Services** → services provided by the node

If you get an error like `Node '/controller_node' not found`, go back to **3.1** and make sure the node is running.

3.3 List all topics

1. In the terminal, type:

```
ros2 topic list
```

2. Press **Enter**.

Expected output example:

```
/cmd_vel  
/imu/data  
/motor_cmd
```

- Shows all active topics in ROS2.
- If your topic is missing, check that the **node publishing it is active** (see section 3.1).

3.4 Inspect a topic

1. Pick a topic (for example `/motor_cmd`).
2. Type:

```
ros2 topic info /motor_cmd
```

3. Press **Enter**.

Expected output:

```
Type: std_msgs/msg/Float32
Publisher count: 1
Subscriber count: 1
```

- **Type** → the kind of message being sent
- **Publisher count** → how many nodes are sending messages on this topic
- **Subscriber count** → how many nodes are listening

If Publisher count = 0 → the node sending messages is not running. Launch it as explained in **3.1**.

3.5 Show message structure

1. To see the contents of a message, type:

```
ros2 interface show std_msgs/msg/Float32
```

2. Press **Enter**.

Output:

```
float32 data
```

- This shows what fields the message has and what type of data is being sent.

3.6 List and inspect services

1. To see all services, type:

```
ros2 service list
```

2. Press **Enter**.
3. To check the type of a service (for example `/reset_pid`), type:

```
ros2 service type /reset_pid
```

4. Press **Enter**.
5. To call the service and see it work:

```
ros2 service call /reset_pid std_srvs/srv/Empty
```

6. Press **Enter**.

If the service is missing → make sure the node providing it is running.

4. Using rqt (GUI Tool)

`rqt` is a graphical tool that lets you **see your nodes and topics visually** instead of using only commands.

4.1 Install rqt

1. Open a terminal.
2. Type the following command:

```
sudo apt install ros-<ros2_distro>-rqt ros-<ros2_distro>-rqt-common-plugins
```

3. Press **Enter**.
4. Enter your computer password if prompted.

4.2 Launch rqt

1. In the terminal, type:

```
rqt
```

2. Press **Enter**.
3. A GUI window will open. You can now **visualize nodes, topics, and messages**.

If no nodes appear, check that your nodes are running (see 3.1) and click **Plugins** → **Refresh**.

4.3 Useful rqt Plugins

Plugin	What it does
rqt_graph	Shows the connections between nodes and topics
rqt_plot	Plots numerical topic data over time
rqt_topic	Views real-time topic messages
rqt_logger_level	Changes node logging level dynamically
rqt_publisher	Lets you publish messages manually

Start with **rqt_graph** to see all nodes and topics visually.

5. Using rqt_graph

`rqt_graph` shows a **live map** of all nodes, topics, and data flow in your ROS2 system.

5.1 Launch rqt_graph

1. Open a terminal.
2. Type:

```
rqt_graph
```

3. Press **Enter**.
4. A window will open showing rectangles (nodes), circles (topics), and arrows (data flow).

If your nodes do not appear, make sure all nodes are running (see **3.1**) and click **Plugins** → **Refresh**.

5.2 Understand the Graph

Example:

```
[IMU_node] ---> /imu/data ---> [Controller_node] ---> /motor_cmd ---> [Motor_node]
```

- Rectangles = nodes
- Circles = topics
- Arrows = direction of data flow

This shows visually how data moves from IMU → Controller → Motors.

5.3 Debugging with `rqt_graph`

- Missing arrow → node isn't publishing or subscribing correctly → check node status and launch it if needed.
 - Extra arrows → unexpected nodes or topics → verify your code.
 - Node names → ensure consistency between code and `rqt_graph`.
-

6. Beginner Tips

- Start with **2 nodes**: publisher & subscriber
- Inspect them with `ros2 node info` and `ros2 topic info`
- Run `rqt_graph` to see the connections visually
- If a node or topic is missing → launch the node and refresh `rqt_graph`
- Experiment with changing topic names and observe the effects live

Think of it like watching the robot's nervous system in action.

7. Real-World Example

Self-Balancing Robot:

- Nodes: `imu_node`, `controller_node`, `motor_node`
- Topics: `/imu/data`, `/motor_cmd`

Check:

1. Run `ros2 node list` → are all nodes running?
 - If not → launch missing nodes (see 3.1)
2. Run `ros2 node info /controller_node` → receives `/imu/data` and publishes `/motor_cmd`?
3. Open `rqt_graph` → arrows showing proper connections?

This confirms your robot "brain" is connected correctly.

8. Summary

After Day 3, you can:

- Open a terminal and **list or inspect nodes**
 - Open a terminal and **check topics and services**
 - Launch missing nodes if required
 - Use **rqt** to visualize nodes and messages
 - Use **rqt_graph** to understand the communication graph
 - Debug connections visually
 - Understand how nodes and topics interact in real-time
-